

从概念、公式到可运行代码

深度学习常见术语与 PyTorch 实践手册

是什么、为什么、作用是什么，以及训练与推理中最容易出错的地方

适用对象：正在学习深度学习、使用 PyTorch 写训练代码，或需要快速排查模型问题的读者

资料更新：2026-09-21 | 以 PyTorch 官方文档和经典论文为主要依据

导读 怎样使用这本手册

这不是只背名词的词典，而是一张从数据到模型、从 loss 到梯度、从训练到推理的路线图。每个核心术语都尽量回答三件事：它是什么、为什么存在、在 PyTorch 中如何落地。涉及 API 的地方会给出可直接迁移到项目中的代码片段。

从数据到参数更新



训练时反复执行这条闭环；推理时通常只执行输入 -> 前向传播 -> 输出。

阅读建议：先看第 1-2 章建立计算图和张量形状的直觉，再看第 3-6 章写出可靠训练循环，最后用第 7-10 章做评估、推理和排错。图中的代码默认使用 PyTorch 2.x 风格；具体 API 仍应以当前版本官方文档为准。

全书反复出现的四个问题

当模型结果不对时，依次问：1）输入和标签的形状、dtype、范围对吗？2）前向输出和损失函数匹配吗？3）梯度真的算出来并更新了吗？4）评估时是否切换了模式且没有数据泄漏？这四问能覆盖大量初学者错误。

目录

导读 怎样使用这本手册 2

目录 3

1 深度学习的整体心智模型 4

2 张量、自动微分与 PyTorch 基础 9

3 数据管线与输入质量 13

4 模型层、结构与表示学习 17

5 输出语义与损失函数 23

6 训练循环与优化工程 27

7 评估、推理与模型保存 33

8 主流网络结构一览 38

9 完整 PyTorch 分类示例 41

9.1 训练与验证模板 42

9.2 推理代码 43

9.3 如果训练不收敛，按这个顺序排查 44

10 高频易错点总清单 45

10.1 概念层面的错误 45

10.2 训练过程的错误 45

10.3 代码层面的错误 46

附录 PyTorch 高频 API 速查与参考资料 47

A. 高频 API 速查 47

B. 参考资料与检索来源 47

1 深度学习的整体心智模型

先把训练过程看成一个闭环：数据进入模型得到预测，预测与标签比较得到损失，损失沿计算图反向传播得到梯度，优化器利用梯度更新参数。大多数术语都可以放回这条闭环中理解。

资料提示

本章概念与 PyTorch autograd 的解释参考 [P1]、[P2]。

人工智能、机器学习、深度学习（AI / ML / DL）

是什么：人工智能是目标领域；机器学习是让系统从数据中学习规则的方法；深度学习是使用多层可学习表示和梯度优化的一类机器学习方法。

为什么需要：把层级关系分清，才能知道问题到底是模型结构、优化算法还是数据问题。深度学习并不等于所有机器学习，也不等于只要网络更深就一定更好。

作用与判断方式：面对一个任务时，先确定学习范式：监督学习需要标签，无监督/自监督可能从数据本身构造目标，强化学习则依靠奖励信号。PyTorch 主要提供张量计算、自动微分和模型训练所需的积木。

易错点

概念误区：把“深度”理解成单纯增加层数；把训练集效果好理解成模型学到了可泛化规律；把模型库、训练框架和算法本身混为一谈。

样本、特征、标签（sample / feature / label）

是什么：样本是一条训练记录；特征是输入 x；标签是期望输出 y。图像、文本、语音或表格都要先变成模型可处理的数值张量。

为什么需要：监督学习的优化目标需要知道模型输出和目标之间的差距。如果标签编码、类别顺序或输入预处理错了，后面的梯度再准确也只是在优化错误目标。

作用与判断方式：把一个 batch 打印出来并检查 shape、dtype、数值范围和标签分布，是训练前最值得做的几分钟检查。

PyTorch pattern

```
x, y = next(iter(train_loader))
print(x.shape, x.dtype, x.min().item(), x.max().item())
print(y.shape, y.dtype, y.unique(return_counts=True))
```

易错点

代码易错：把类别标签做成 one-hot 后直接传给 CrossEntropyLoss；把图像值保留在 0-255，却按已归一化数据的均值方差处理；标签 dtype 不符合损失函数要求。

模型、参数与超参数（model / parameter / hyperparameter）

是什么：模型是从输入到输出的函数；参数是训练过程中被梯度更新的权重和偏置；超参数是训练者设定的配置，例如学习率、batch size、层数、隐藏维度和 weight decay。

为什么需要：参数表达模型从数据中学到的内容，超参数决定学习过程和模型容量。区分两者可以避免把学习率误当成模型参数，也能正确保存和复现实验。

作用与判断方式：在 PyTorch 中，注册为 nn.Parameter 的张量会被 model.parameters() 返回；优化器只会更新传入参数组中的参数。

PyTorch pattern

```
for name, p in model.named_parameters():
    print(name, p.shape, p.requires_grad)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
```

易错点

概念误区：requires_grad=True 不等于“它一定会更新”；参数还必须被模型注册、传给优化器、拥有非空梯度，并且优化器 step 被执行。

前向传播（forward pass）

是什么：给定输入 x，模型按网络结构逐层计算预测值 y_hat 的过程。前向传播不仅产生输出，也会在启用 autograd 时记录计算图。

为什么需要：训练需要根据预测构造损失，推理则只需要执行前向传播。前向输出的 shape 和语义决定损失函数能否正确计算。

作用与判断方式：model(x) 会调用 forward()；不要直接手动调用 model.forward(x)，因为模块调用还会处理 hooks 等机制。

PyTorch pattern

```
logits = model(inputs)       # forward pass
loss = loss_fn(logits, target)
```

易错点

代码易错：把模型输出经过了 softmax 后再传给 CrossEntropyLoss；或者分类输出维度写成 1，却用多分类损失。先明确输出是 logits、概率还是连续值。

损失函数（loss function / objective）

是什么：损失函数把预测和目标映射成一个标量或可约简的误差值。训练通常最小化经验风险：整个数据集上样本损失的平均或加权平均。

为什么需要：优化器需要一个可微的目标，损失函数把任务目标转成了可通过梯度下降处理的数值。不同任务、标签编码和输出语义需要不同损失。

作用与判断方式：训练时记录 loss 是为了观察优化过程，但最终业务目标不一定等于 loss；分类任务仍应同时看 accuracy、F1、召回率等指标。

公式 / 直觉

以均方误差为例：L = mean((y_hat - y)^2)。损失小表示预测在该目标下更接近标签，但不保证所有业务指标都好。

易错点

概念误区：把 loss 当成准确率；只看训练 loss 不看验证集；对每个 batch 的 loss 做简单平均，却忽略了最后一个 batch 的样本数不同。

梯度（gradient）

是什么：梯度是损失对参数的偏导数，表示参数微小变化时损失如何变化。对单个参数 θ，梯度是 ∂L/∂θ；对全部参数组合起来就是梯度向量/张量。

为什么需要：梯度提供了局部下降方向：沿负梯度移动通常能让损失下降。它是自动微分与优化器之间的接口。

作用与判断方式：训练中查看 p.grad 可以判断反向传播是否覆盖了某个参数；梯度为 None、全 0、NaN 或极大值，分别提示不同问题。

PyTorch pattern

```
loss.backward()
for name, p in model.named_parameters():
    if p.grad is not None:
        print(name, p.grad.norm().item())
```

易错点

代码易错：把 p.grad 当成更新后的参数；梯度默认会累积，连续多个 batch 不清零会改变有效梯度；非叶子张量通常不会自动保留 grad。

反向传播（backpropagation）

是什么：反向传播是沿计算图从损失向输入方向使用链式法则计算梯度的过程。在 PyTorch 中，loss.backward() 触发 autograd 遍历图并把梯度累积到叶子参数的 .grad。

为什么需要：手工推导每一层导数在复杂网络中成本很高，自动微分让我们只写前向计算，框架自动完成局部导数的组合。

作用与判断方式：反向传播本身只计算梯度，不负责更新参数；更新由 optimizer.step() 完成。标量 loss 默认相当于对它反向传播一个系数为 1 的梯度。

公式 / 直觉

链式法则：若 L 依赖 a，a 依赖 z，z 依赖 w，则 $\partial L/\partial w = (\partial L/\partial a)(\partial a/\partial z)(\partial z/\partial w)$。

PyTorch pattern

```
optimizer.zero_grad(set_to_none=True)
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()       # ████████
optimizer.step()      # ████████
```

易错点

概念易错：反向传播不是“把参数倒着传回去”；它是计算导数的过程。代码易错：忘记 zero_grad()、对同一张图重复 backward、在 torch.no_grad() 中训练。

优化器与参数更新（optimizer / update）

是什么：优化器根据参数当前值、梯度以及自身状态计算下一步参数。最基本的梯度下降是 θ <- θ - η∇θL，其中 η 是学习率。

为什么需要：同一个梯度在不同学习率、动量或自适应算法下会产生不同更新轨迹。优化器决定“如何走”，损失函数决定“往哪里走”。

作用与判断方式：常见优化器包括 SGD、SGD+momentum、Adam、AdamW。选择优化器后，还要确认参数组、学习率和 weight decay 的含义。

PyTorch pattern

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
# ■■optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
optimizer.step()
```

易错点

训练易错：学习率太大导致 loss 发散，太小导致几乎不动；恢复 checkpoint 时只加载模型权重却不加载优化器状态，导致续训轨迹改变。

训练、验证、测试与推理（train / validation / test / inference）

是什么：训练集用于更新参数；验证集用于选择超参数和模型版本；测试集用于最终一次性估计泛化能力；推理是把已训练模型用于新输入的前向计算。

为什么需要：如果反复根据测试集结果调参，测试集就不再是独立的最终评估。验证阶段不应更新模型参数，也不应把验证样本统计量泄漏到训练流程。

作用与判断方式：用清晰的生命周期管理数据和模型：训练只看训练集，调参看验证集，最后报告测试集；部署推理要固定预处理、标签映射和模型版本。

易错点

最常见误区：把测试集当验证集反复试；验证时忘记 model.eval()；推理预处理与训练预处理不一致；保存了模型却没有保存类别到索引的映射。

2 张量、自动微分与 PyTorch 基础

PyTorch 的核心对象是 Tensor。很多“模型不工作”的根因不是网络思想，而是 shape、dtype、device 或计算图状态不一致。

资料提示

张量、自动微分和模型状态的定义参考 [P1]、[P3]、[P4]。

图像张量形状：批次 B、通道 C、高 H、宽 W

[B, C, H, W]	[B, H, W, C]	[B, T, D]
PyTorch CNN 常见	部分图像库常见	序列/Transformer 常见

形状不是装饰：它决定矩阵乘法、卷积、拼接和损失函数是否能正确对齐。

张量与形状（tensor / shape）

是什么：张量是带有任意维度的数值容器；shape 描述每个维度大小。例如图像 batch 常见为 [B, C, H, W]，文本序列常见为 [B, T, D]。

为什么需要：矩阵乘法、卷积、拼接和归约都依赖维度语义。只看元素数量相同是不够的，维度顺序错了也可能得到能运行但语义错误的结果。

作用与判断方式：建立项目自己的 shape 约定，并在关键边界打印或断言。把 B、C、H、W、T、D 的含义写在注释或文档中。

PyTorch pattern

```
assert images.ndim == 4
B, C, H, W = images.shape
features = images.flatten(1)  # ■■ batch ■
```

易错点

形状易错：view(-1) 把 batch 维也展平；squeeze() 在 batch size=1 时可能把 batch 维误删；CNN 输入把 HWC 直接传给需要 CHW 的层。

dtype、device 与数据移动（dtype / device）

是什么：dtype 表示数据精度和类型，如 float32、float16、bfloat16、int64；device 表示张量所在设备，如 CPU 或 CUDA。

为什么需要：设备决定计算在哪里发生，dtype 影响精度、速度、显存和算子兼容性。模型参数与输入通常必须在同一设备上，标签 dtype 还受损失函数约束。

作用与判断方式：先定义统一 device，再把模型和每个 batch 显式移动过去；分类标签通常使用 torch.long，连续值输入通常使用浮点类型。

PyTorch pattern

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
inputs = inputs.to(device, non_blocking=True)
target = target.to(device)
```

易错点

代码易错：只把模型放到 GPU，忘记把 batch 放过去；把 label 变成 float 导致 CrossEntropyLoss 报错；混合精度下手动把所有权重强行转成 float16，造成数值问题。

广播机制（broadcasting）

是什么：当两个张量维度从右往左相容时，PyTorch 会在大小为 1 的维度上逻辑扩展它们，使逐元素运算能够进行。

为什么需要：广播让归一化、偏置加法和批量计算更简洁，但也可能隐藏维度错误，让程序运行却产生意外结果。

作用与判断方式：理解广播规则：从末维开始比较，维度相等或其中一个为 1 才能广播；广播不会真的复制数据，但会改变运算语义。

PyTorch pattern

```
x = torch.randn(4, 3)
bias = torch.randn(3)
y = x + bias          # [4, 3] + [3]
```

易错点

概念易错：[B, 1] 与 [B] 广播后常常得到 [B, B]，尤其在计算回归损失时会悄悄扩大维度。出现异常时先打印 prediction.shape 和 target.shape。

reshape、view、permute 与 contiguous（shape transforms）

是什么：reshape 试图返回目标形状；view 要求底层内存布局适合视图；permute 重新排列维度但通常返回非连续张量；contiguous 把它变成连续布局。

为什么需要：卷积、序列和多头注意力经常需要在不同维度顺序之间切换。正确区分这些操作能避免运行时错误和隐藏的维度语义错误。

作用与判断方式：优先使用语义明确的操作，并在 permute 后需要 view 时调用 contiguous()，或直接使用 reshape()。

PyTorch pattern

```
x = torch.randn(2, 3, 4)
y = x.permute(0, 2, 1)        # [2, 4, 3]
z = y.contiguous().view(2, -1)
```

易错点

代码易错：把 transpose/permute 后的张量直接 view；误以为 reshape 永远不复制；改变维度顺序后忘记同步修改后续层的输入语义。

requires_grad、计算图与叶子节点（autograd graph）

是什么：当张量参与需要梯度的运算时，PyTorch 会构建动态计算图。叶子参数通常是用户创建且 requires_grad=True 的张量；中间结果带有 grad_fn。

为什么需要：动态图能记录本轮前向的操作，并在 backward 时按反向拓扑顺序计算梯度。每个 iteration 的图通常在 backward 后释放，以节省内存。

作用与判断方式：查看 tensor.requires_grad、tensor.grad_fn 和参数的 .grad 是理解自动微分状态的三种常用手段。

PyTorch pattern

```
w = torch.randn(3, requires_grad=True)
z = (w * 2).sum()
print(w.is_leaf, z.grad_fn is not None)
z.backward()
print(w.grad)
```

易错点

代码易错：在循环中保存带计算图的 loss 或输出，造成显存不断增长；想对中间张量读 .grad 却没有调用 retain_grad()；原地操作破坏 autograd 所需的版本信息。

detach、no_grad 与 inference_mode（gradient modes）

是什么：detach() 返回与原张量共享数据但脱离计算图的张量；torch.no_grad() 在上下文中关闭梯度记录；torch.inference_mode() 是更强的推理模式，适用于确定不会与 autograd 交互的场景。

为什么需要：推理和特征导出不需要反向图，关闭梯度可降低显存和计算开销。detach 则适合切断某个分支的梯度流。

作用与判断方式：训练冻结主干、保存预测、转 numpy 或做评估时要选择正确方式：关闭上下文是全局作用域控制，detach 是对单个结果切断。

PyTorch pattern

```
model.eval()
with torch.inference_mode():
    logits = model(x)
    pred = logits.argmax(dim=1).cpu()

features = encoder(x).detach()
```

易错点

概念易错：model.eval() 不等于关闭梯度；no_grad() 不等于切换 Dropout/BatchNorm；detach() 不会复制数据，后续原地修改可能影响共享存储。

nn.Module、Parameter 与 buffer（module / parameter / buffer）

是什么：nn.Module 组织子层并提供参数注册、模式切换和设备迁移；nn.Parameter 表示应被优化器看到的可训练参数；buffer 是随模型保存和迁移但不参与优化的状态，例如 BatchNorm 的 running mean。

为什么需要：正确注册对象，才能让 model.parameters()、state_dict()、.to(device) 和 train/eval 递归生效。

作用与判断方式：写自定义模型时把层放在 __init__，在 forward 中定义数据流；固定但需要保存的张量用 register_buffer。

PyTorch pattern

```
class Net(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = torch.nn.Linear(10, 2)
        self.register_buffer("scale", torch.ones(1))

    def forward(self, x):
        return self.fc(x) * self.scale
```

易错点

代码易错：把层放进普通 Python list，导致参数没有注册；忘记调用 super().__init__()；把需要学习的张量写成普通 Tensor，优化器看不到它。

3 数据管线与输入质量

数据决定上限，训练循环决定能否接近上限。Dataset 负责如何取一条样本，DataLoader 负责如何组成 batch 和迭代，transform 负责输入变换。

资料提示

Dataset/DataLoader/transform 的官方示例参考 [P5]、[P6]。

Dataset（dataset）

是什么：Dataset 定义“给定索引，如何返回一个样本”。通常实现 __len__ 和 __getitem__，返回输入与标签，或一个包含多个字段的字典。

为什么需要：把样本读取逻辑与模型解耦，可以更换文件格式、缓存策略和数据增强，而不修改模型。

作用与判断方式：检查 len(dataset)、单样本类型、每个字段的 shape 和 label 编码，再交给 DataLoader。

PyTorch pattern

```
class MyDataset(torch.utils.data.Dataset):
    def __init__(self, xs, ys, transform=None):
        self.xs, self.ys, self.transform = xs, ys, transform

    def __len__(self):
        return len(self.xs)

    def __getitem__(self, idx):
        x, y = self.xs[idx], self.ys[idx]
        return self.transform(x) if self.transform else x, y
```

易错点

代码易错：__len__ 与样本数不一致；训练和验证 Dataset 共享了随机增强；读取文件时标签顺序和类别映射发生变化。

DataLoader、batch 与 shuffle（data loader / mini-batch）

是什么：DataLoader 迭代 Dataset，负责 batch 组装、打乱、并行加载、丢弃不完整 batch 等。mini-batch 是一次计算梯度所用的一组样本。

为什么需要：一次处理全部数据通常显存不可承受，逐样本梯度又噪声大且效率低；mini-batch 是计算效率、梯度稳定性和显存之间的折中。

作用与判断方式：训练集通常 shuffle=True；验证/测试通常不需要打乱。batch_size 影响显存、吞吐、梯度噪声和有效学习率。

PyTorch pattern

```
train_loader = DataLoader(
    train_set, batch_size=64, shuffle=True,
    num_workers=4, pin_memory=True, drop_last=False
)
```

易错点

训练易错：验证集也使用随机增强或 shuffle 导致结果难复现；drop_last=True 使少量样本永远不参与训练；多进程 DataLoader 在 Windows 下忘记入口保护。

预处理、归一化与标准化（preprocessing / normalization）

是什么：预处理把原始数据转换为模型期望的数值范围和布局。常见标准化为 x' = (x - mean) / std，图像还常将 [0,255] 缩放到 [0,1]。

为什么需要：神经网络对输入尺度敏感。尺度一致可改善条件数、加快优化，并使预训练模型的输入分布与训练时保持一致。

作用与判断方式：均值、方差、词表、类别映射等统计量只能用训练集估计，再固定应用到验证、测试和线上数据。

PyTorch pattern

```
mean = train_x.mean(dim=0, keepdim=True)
std = train_x.std(dim=0, keepdim=True).clamp_min(1e-6)
train_x = (train_x - mean) / std
valid_x = (valid_x - mean) / std
```

易错点

概念易错：用全数据计算均值方差造成泄漏；对已经标准化的预训练模型输入重复标准化；图像通道顺序、颜色空间与训练时不同。

数据增强（data augmentation）

是什么：数据增强对训练样本施加保持标签语义的随机变换，例如图像裁剪、翻转、颜色扰动，文本中的同义改写或遮盖。

为什么需要：增强通过改变输入而不是简单复制样本，扩大有效训练分布，减少过拟合并提升对真实变化的鲁棒性。

作用与判断方式：增强必须符合任务不变性：猫图像水平翻转通常合理，但数字识别中翻转可能改变类别；验证和测试通常使用确定性预处理。

PyTorch pattern

```
train_tf = torchvision.transforms.Compose([
    torchvision.transforms.RandomResizedCrop(224),
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor(),
])
```

易错点

训练易错：增强改变了标签语义；把随机增强放到验证/测试；增强过强造成欠拟合；同一张原图的不同增强版本跨越 train/valid 划分造成泄漏。

训练集、验证集、测试集与数据泄漏（split / leakage）

是什么：数据集划分决定哪些样本能够影响参数、超参数和最终报告。数据泄漏是评估信息以不该有的方式进入训练或调参流程。

为什么需要：没有独立评估，就无法知道模型是否能泛化到未见数据。泄漏会让指标虚高，线上表现却突然下降。

作用与判断方式：按实体、用户、时间或视频片段做合理分组；同一主体的近重复样本不应被随机拆到不同集合。任何用全数据计算的统计量都要警惕泄漏。

易错点

概念易错：把随机切分当成永远正确；医疗/用户/时间序列数据常需按患者、用户或时间切分；先全量去重、归一化、特征选择再切分也可能泄漏验证信息。

padding、mask 与变长序列（padding / mask）

是什么：批处理要求同一 batch 内张量形状通常一致，因此把短序列补到统一长度；mask 标记哪些位置是补齐内容、不可参与注意力或损失。

为什么需要：如果模型把 padding 当成真实 token，平均池化、注意力和损失都会被无意义位置污染。

作用与判断方式：明确 mask 的语义和 shape；注意 PyTorch 中 attention mask 可能是 bool 或加性 -inf mask，不同 API 的约定不同。

PyTorch pattern

```
tokens = torch.nn.utils.rnn.pad_sequence(
    sequences, batch_first=True, padding_value=pad_id
)
valid_mask = tokens.ne(pad_id)  # [B, T]
```

易错点

代码易错：mask 取反；把 [B,T] mask 直接与 [B,H,T,T] 注意力分数相加却未扩维；计算 token loss 时忘记忽略 padding。

类别不平衡、采样与 class weight（imbalance）

是什么：类别不平衡表示不同类别的样本量或错误代价差异很大。模型可能只预测多数类，却得到看似不错的 accuracy。

为什么需要：训练目标和采样分布决定梯度主要来自谁。少数类召回率差时，需要调整损失权重、采样策略、阈值或数据收集方案。

作用与判断方式：结合混淆矩阵、每类 precision/recall/F1 和 PR-AUC 评估；不要只看总体准确率。

PyTorch pattern

```
weights = torch.tensor([1.0, 4.0, 2.0], device=device)
loss_fn = torch.nn.CrossEntropyLoss(weight=weights)
```

易错点

概念易错：class weight 不是万能修复；过度重复少数类可能过拟合；加权训练后仍需在真实业务分布上评估，不能只报告加权指标。

4 模型层、结构与表示学习

模型层不是孤立 API，而是对输入做某种归纳偏置：Linear 混合特征，卷积利用局部平移结构，注意力动态选择上下文，归一化和残差帮助深层网络训练。

资料提示
经典结构和官方层文档参考 [P7]-[P13] 及参考文献 [R1]-[R7]。

Linear / 全连接层（linear layer）

是什么：Linear 层做 y = xW^T + b，把最后一维从 in_features 映射到 out_features。它是 MLP、分类头、投影层和注意力中的基本构件。

为什么需要：线性层能学习特征的线性组合，叠加非线性激活后可以逼近复杂函数。

作用与判断方式：确认输入最后一维等于 in_features；批次和前置维度可以保留，Linear 只作用于最后一维。

PyTorch pattern

```
layer = torch.nn.Linear(in_features=128, out_features=10)
x = torch.randn(32, 128)
logits = layer(x)  # [32, 10]
```

易错点
维度易错：把 [B,C,H,W] 直接传给期望最后一维为 in_features 的 Linear；flatten 时误删 batch 维；输出类别数与标签类别数不一致。

卷积、步幅、padding 与感受野（convolution / stride / padding / receptive field）

是什么：卷积核在局部窗口上共享参数并滑动计算，stride 控制移动步长，padding 控制边界补齐，感受野表示一个输出位置能看到的输入区域。

为什么需要：卷积利用图像局部性和参数共享，参数量与计算量通常比同规模全连接更低；堆叠卷积可扩大感受野。

作用与判断方式：2D CNN 常用输入 [B,C,H,W]，输出通道由 out_channels 决定。输出空间尺寸受 kernel、stride、padding、dilation 共同决定。

PyTorch pattern

```
conv = torch.nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1)
x = torch.randn(8, 3, 224, 224)
y = conv(x)  # [8, 32, 112, 112]
```

易错点
代码易错：把 NHWC 输入传给 NCHW 卷积；忘记 stride=2 会改变空间大小；padding 和标签的像素对齐不一致；对卷积核与输出通道含义混淆。

激活函数（activation）

是什么：激活函数给线性层引入非线性。常见有 ReLU、GELU、Sigmoid、Tanh 和 Softmax。

为什么需要：没有非线性时，多层线性层仍可合并成一层线性变换；激活函数决定梯度形状、饱和区间和训练稳定性。

作用与判断方式：隐藏层常用 ReLU/GELU；二分类、多标签输出常在损失内部使用 sigmoid；多分类输出通常交给 CrossEntropyLoss 处理。

PyTorch pattern

```
x = torch.nn.functional.gelu(x)
# ■■■■■■■■■■■ logits■■■■■■■■■■ softmax
logits = classifier(features)
```

易错点

概念易错：把 Softmax 当成隐藏层激活；把 Sigmoid 用在互斥多分类；ReLU 死亡神经元过多；推理时概率和训练时 logits 的语义混用。

池化与下采样（pooling / downsampling）

是什么：最大池化或平均池化在局部窗口内做聚合，减少空间尺寸和计算量；现代网络也常用 stride convolution 实现下采样。

为什么需要：下采样扩大相对感受野、减少特征图大小，并提供一定局部不变性。

作用与判断方式：选择池化方式要看任务：分类通常可以更激进，下游分割/检测需要保留空间细节。

易错点

概念易错：把池化理解成“无损压缩”；下采样会丢信息；分类头前直接固定 Linear 输入大小，改输入分辨率后 shape 崩溃。

参数初始化（weight initialization）

是什么：初始化给参数提供训练起点。常见 Xavier/Glorot 让前后层方差较平衡，He/Kaiming 针对 ReLU 类激活调整尺度。

为什么需要：初始化过大可能梯度爆炸，过小可能梯度消失；好的初始化让信号和梯度在深层传播时更稳定。

作用与判断方式：PyTorch 层通常有默认初始化，但自定义层、迁移学习和特殊激活可能需要显式初始化；初始化要在优化器创建前完成。

PyTorch pattern

```
def init_weights(m):
    if isinstance(m, torch.nn.Linear):
        torch.nn.init.kaiming_normal_(m.weight)
        torch.nn.init.zeros_(m.bias)

model.apply(init_weights)
```

易错点

代码易错：初始化后才创建优化器通常没问题，但替换 Parameter 或层后旧优化器可能仍指向旧参数；初始化漏掉 bias；对 LayerNorm/BatchNorm 使用不合适的初始化。

Dropout 与正则化（dropout / regularization）

是什么：Dropout 在训练时随机把部分激活置零，并按保留概率进行缩放；推理时关闭随机失活。它是一种减少特征共适应的正则化方法。

为什么需要：随机失活相当于训练许多稀疏子网络并近似集成，常用于降低过拟合。

作用与判断方式：Dropout 是模型行为而非优化器设置，受 model.train() / model.eval() 控制；强度由 p 决定，过强会导致欠拟合。

PyTorch pattern

```
class Head(torch.nn.Module):
    def __init__(self, d, n_classes):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Dropout(p=0.2),
            torch.nn.Linear(d, n_classes),
        )
```

易错点

训练/推理易错：验证时忘记 eval() 导致预测随机；把 Dropout 当成只在训练脚本里手动调用的函数；p 过大导致训练 loss 和验证 loss 都降不下来。

BatchNorm、LayerNorm 与归一化层（normalization layers）

是什么：BatchNorm 使用 batch 统计量归一化并维护运行均值方差；LayerNorm 在特征维度上对单个样本归一化，不依赖 batch 维度。

为什么需要：归一化可改善优化条件、稳定激活范围，并让更深或更大的网络更容易训练。不同归一化层对 batch size、序列维度和训练/推理状态的依赖不同。

作用与判断方式：CNN 常见 BatchNorm2d；Transformer 常见 LayerNorm/RMSNorm；小 batch 或变长序列场景要谨慎选择。

易错点

概念易错：把 BatchNorm 的 running mean 当成可学习参数；训练和推理模式下统计量来源不同；小 batch 下 BatchNorm 统计不稳定；LayerNorm 的 normalized_shape 写错。

残差连接（residual / skip connection）

是什么：残差块让输出成为 F(x) + x，跳过路径把输入直接加到变换后的结果。

为什么需要：它提供更短的梯度路径，使深层网络更容易优化，也让网络可以学习“在恒等映射上做增量修正”。

作用与判断方式：要求两条路径 shape 兼容；通道数或空间尺寸不同时，需要 projection shortcut，通常是 1x1 卷积。

PyTorch pattern

```
class Block(torch.nn.Module):
    def __init__(self, d):
        super().__init__()
        self.f = torch.nn.Sequential(
            torch.nn.Linear(d, d), torch.nn.ReLU(), torch.nn.Linear(d, d)
        )
    def forward(self, x):
        return x + self.f(x)
```

易错点

代码易错：残差相加前 shape 不一致；在错误位置做激活导致结构语义改变；原地修改 residual 分支破坏反向传播。

Embedding 与表示（embedding）

是什么：Embedding 把离散索引映射为可学习的稠密向量。词、子词、类别、物品或位置都可以用 embedding 表示。

为什么需要：稠密向量让模型通过相似性和上下文学习离散实体的连续表示，比 one-hot 更适合大词表和参数共享。

作用与判断方式：nn.Embedding(num_embeddings, embedding_dim) 的输入通常是 Long 索引，输出会在末尾增加 embedding_dim。

PyTorch pattern

```
emb = torch.nn.Embedding(num_embeddings=30000, embedding_dim=768)
token_ids = torch.randint(0, 30000, (8, 128), dtype=torch.long)
h = emb(token_ids)  # [8, 128, 768]
```

易错点

代码易错：索引不是 long；padding_idx 未设置；词表 ID 越界；序列长度、batch 维和 embedding 维混淆。

Attention 与 Transformer（attention / transformer）

是什么：Attention 根据 Query 与 Key 的相关性生成权重，再对 Value 加权求和。Transformer 以多头自注意力、前馈网络、残差和归一化为核心。

为什么需要：注意力能动态建立远距离依赖，不必像纯 RNN 那样按时间步串行处理；多头机制让模型在不同子空间学习不同关系。

作用与判断方式：文本、视觉和多模态模型大量采用 Transformer；训练时要处理 mask、位置编码、显存复杂度和序列长度。

公式 / 直觉

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V。除法用于抑制点积随维度增大而过大的问题。

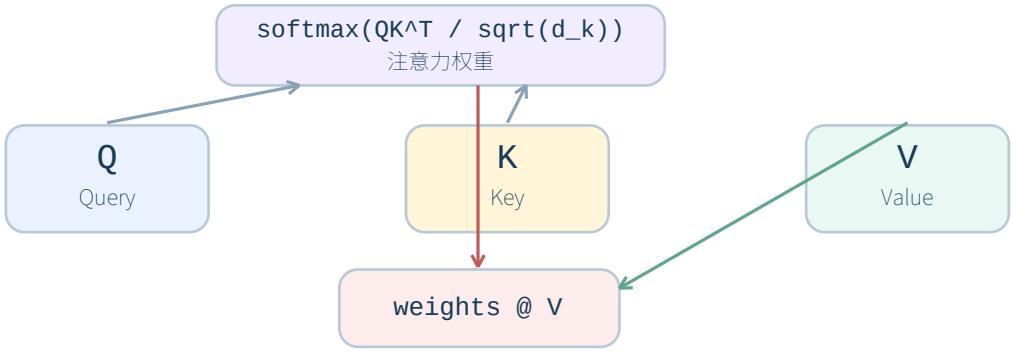
PyTorch pattern

```
attn = torch.nn.MultiheadAttention(
    embed_dim=256, num_heads=8, batch_first=True
)
out, weights = attn(query, key, value, key_padding_mask=pad_mask)
```

易错点

概念易错：把 attention 权重当成严格因果解释；忽略 mask 方向；序列长度翻倍时注意力矩阵通常按平方增长；输入 layout [T,B,D] 与 batch_first=True 的 [B,T,D] 混用。

Scaled dot-product attention：用 Q 与 K 计算相关性，再对 V 加权



Multi-head attention 只是把这件事并行做多次，再拼接并投影。

5 输出语义与损失函数

损失函数必须与任务类型、标签编码和模型输出严格匹配。最常见的错误不是“模型太差”，而是 logits、概率、类别索引和 one-hot 标签配错。

资料提示

损失函数的参数语义参考 PyTorch 官方 CrossEntropyLoss、BCEWithLogitsLoss 和 MSELoss 文档 [P14]-[P16]。

回归与 MSE / MAE（regression / MSE / MAE）

是什么：回归预测连续值。MSE 对误差平方，惩罚大误差更强；MAE 对绝对误差，通常对离群点更稳健。

为什么需要：目标函数体现错误代价：如果大偏差特别危险，MSE 更敏感；如果数据有离群点或希望更稳健，MAE/Huber 可能更合适。

作用与判断方式：检查预测和标签的 shape 是否一致，并注意目标是否需要标准化；最终指标应还原到业务单位。

PyTorch pattern

```
loss_mse = torch.nn.MSELoss()
loss_mae = torch.nn.L1Loss()
loss = loss_mse(prediction, target)
```

易错点

代码易错：prediction 为 [B,1]、target 为 [B] 时广播成 [B,B]；在标准化目标上训练后忘记反标准化；用 accuracy 评价连续值预测。

logits、概率与决策（logits / probability / decision）

是什么：logits 是未归一化的模型分数；概率是经过 sigmoid 或 softmax 后的值；decision 是阈值或 argmax 后的离散预测。

为什么需要：把三者分清能正确选择损失、指标和线上阈值。训练通常保留 logits 以获得更稳定的数值计算，推理/展示时才转概率。

作用与判断方式：多分类 logits shape 常为 [B,C]；二分类可用 [B] 或 [B,1]；多标签通常为 [B,C]，每个类别独立使用 sigmoid。

PyTorch pattern

```
logits = model(x)
prob = torch.softmax(logits, dim=1)
pred_class = logits.argmax(dim=1)
```

易错点

概念易错：logits 可能是负数且不要求和为 1；概率阈值 0.5 并不适用于所有类别；把 argmax 的类别索引当成概率。

CrossEntropyLoss 与多分类（cross entropy）

是什么：多分类交叉熵接收每个类别的 logits 和每个样本的类别索引，内部完成 log-softmax 与负对数似然。

为什么需要：它用正确类别的相对分数衡量模型置信度，数值实现通常比手动 softmax 再取 log 更稳定。

作用与判断方式：输入 logits shape [B,C,...]；target 通常是 Long 类型的类别索引，范围为 [0,C)。模型最后一层不要手动 Softmax。

公式 / 直觉

单样本可理解为 L = -log softmax(logits)[target]。目标类别概率越低，损失越大。

PyTorch pattern

```
loss_fn = torch.nn.CrossEntropyLoss()
logits = classifier(features)        # [B, C], no softmax
target = target.long()               # [B]
loss = loss_fn(logits, target)
```

易错点

三大易错：1）先 softmax 再传入 CrossEntropyLoss；2）target 用 one-hot/float；3）类别数 C、标签最大值和类别映射不一致。

BCEWithLogitsLoss 与二分类/多标签（binary cross entropy with logits）

是什么：BCEWithLogitsLoss 把 sigmoid 与二元交叉熵合在一起。二分类可输出一个 logit；多标签分类可输出 C 个彼此独立的 logits。

为什么需要：合并实现有更好的数值稳定性，尤其是极大或极小 logit。多标签问题不应使用互斥的 softmax。

作用与判断方式：target 通常是 float，shape 与 logits 一致，值在 0 到 1；多标签每一列表示一个独立标签。

PyTorch pattern

```
loss_fn = torch.nn.BCEWithLogitsLoss()
logits = classifier(features).squeeze(-1)  # [B]
target = target.float()                    # [B]
loss = loss_fn(logits, target)

prob = torch.sigmoid(logits)
pred = (prob >= 0.5).long()
```

易错点

代码易错：用 CrossEntropyLoss 处理多标签；target 是 long 且 shape 不一致；对 BCEWithLogitsLoss 前手动 sigmoid；二分类输出 [B,2] 却又用 BCE。

Softmax、Sigmoid 与维度（softmax / sigmoid）

是什么：Sigmoid 对每个 logit 独立映射到 (0,1)；Softmax 在指定维度上把一组 logits 变成和为 1 的分布。

为什么需要：Sigmoid 适合独立二元事件，多标签需要多个 sigmoid；Softmax 适合互斥多分类。dim 决定概率归一化沿哪个维度进行。

作用与判断方式：训练损失通常使用 logits 版本；推理时指定正确 dim，并在 batch、序列或类别维度之间分清语义。

PyTorch pattern

```
multi_class_prob = torch.softmax(logits, dim=1)  # classes
multi_label_prob = torch.sigmoid(logits)          # labels independently
```

易错点

维度易错：对 [B,C] 用 softmax(dim=0) 会在 batch 内归一化；对序列 logits 用错维度会让 token 之间而不是类别之间竞争。

reduction、class weight 与 label smoothing（reduction / weighting）

是什么：损失可以返回每个样本的值（none），也可以求 mean 或 sum；class weight 调整类别贡献；label smoothing 把硬标签变得略平滑。

为什么需要：reduction 决定梯度尺度，weight 决定不同样本/类别对目标的相对影响。标签平滑可降低过度自信，但会改变目标定义。

作用与判断方式：做梯度累积时要特别注意 mean loss 的缩放；比较不同 batch size 的 loss 时，确认 reduction 和样本权重一致。

PyTorch pattern

```
loss_fn = torch.nn.CrossEntropyLoss(
    label_smoothing=0.1, reduction="mean"
)
per_sample = torch.nn.CrossEntropyLoss(reduction="none")(logits, target)
```

易错点

概念易错：sum 与 mean 的梯度大小不同；用 class weight 后仍用未经加权的 accuracy 做唯一结论；label smoothing 不会修复错误标签或数据泄漏。

多任务与复合损失（multi-task / composite loss）

是什么：一个模型同时优化多个目标，例如分类 + 回归、检测分类 + 框回归。总损失常是各分支损失的加权和。

为什么需要：多个任务可以共享表示并互相提供监督，但不同损失的数值尺度和梯度强度可能互相竞争。

作用与判断方式：记录每个子损失和加权后的总损失；先让每个分支的 shape、标签和单独训练都正确，再调权重。

PyTorch pattern

```
loss_cls = ce(cls_logits, cls_target)
loss_box = smooth_l1(box_pred, box_target)
loss = loss_cls + 2.0 * loss_box
```

易错点

训练易错：某个损失大几个数量级压制其它任务；只打印总 loss 找不到是哪一项异常；不同任务的 batch mask 没有正确处理。

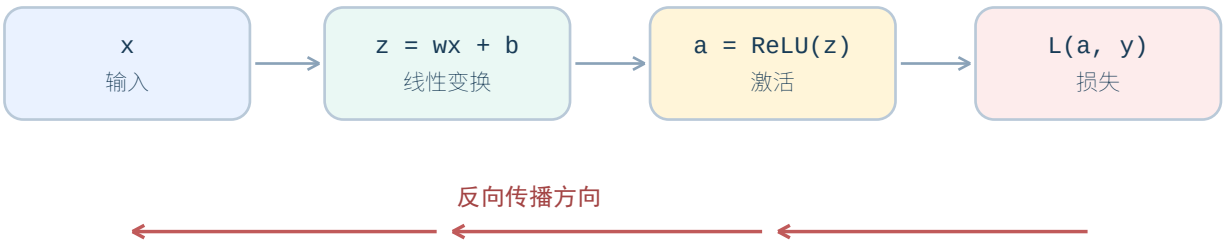
6 训练循环与优化工程

这一章是 PyTorch 实战核心。先写出正确、可观察的最小训练循环，再考虑学习率调度、混合精度、梯度累积和分布式训练。

资料提示

训练与优化示例参考 [P17]-[P20]，混合精度细节参考 [P21]。

计算图与链式法则：为什么 loss.backward() 能得到每个参数的梯度



每条边提供一个局部导数，autograd 按链式法则把它们相乘并累积到叶子参数的 .grad。

一个正确的最小训练循环（training loop）

是什么：训练循环按 batch 执行：清空旧梯度，前向计算，计算 loss，反向传播，优化器更新。epoch 是遍历一遍训练集，step 通常指一次 optimizer 更新。

为什么需要：把顺序写对，才能保证每次更新使用当前 batch 的梯度且不把历史梯度混进来。

作用与判断方式：建议先让一个小 batch 过拟合，确认 loss 能下降，再扩大数据和模型；把训练、验证、记录和保存拆成可测试函数。

PyTorch pattern

```
for epoch in range(num_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)
        logits = model(x)
        loss = loss_fn(logits, y)
        loss.backward()
        optimizer.step()
```

易错点

顺序易错：忘记 zero_grad()；先 step() 再 backward()；在 epoch 外只更新一次；把验证代码放进训练图；把 loss.item() 当作可反向传播的 loss。

zero_grad 与梯度累积（zero_grad / accumulation）

是什么：PyTorch 的梯度默认累积到 .grad，zero_grad() 清理上一步梯度。梯度累积则故意跨多个 micro-batch 累加，再进行一次 optimizer.step。

为什么需要：累积可以在显存不足时模拟更大的有效 batch，也解释了为什么普通训练必须清零。

作用与判断方式：若每个 micro-batch 的 loss 是 mean，累积前通常除以 accumulation_steps，使更新尺度与目标有效 batch 对齐。

PyTorch pattern

```
optimizer.zero_grad(set_to_none=True)
for i, (x, y) in enumerate(loader):
    with torch.autocast(device_type=device.type, enabled=use_amp):
        loss = loss_fn(model(x), y) / accum_steps
    scaler.scale(loss).backward()
    if (i + 1) % accum_steps == 0:
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)
```

易错点

代码易错：累积时忘记除以步数，梯度变大；最后不足一个 accumulation window 的梯度没 step；每个 micro-batch 都 step，实际上没有累积。

SGD、Adam 与 AdamW（optimizers）

是什么：SGD 沿梯度下降，可加 momentum；Adam 使用一阶、二阶梯度矩估计做自适应步长；AdamW 把 decoupled weight decay 从梯度更新中分离。

为什么需要：不同优化器的噪声、收敛速度、泛化和超参数敏感性不同。Adam/AdamW 常作为稳定起点，SGD 在视觉任务中仍常见。

作用与判断方式：优化器不是越复杂越好；比较实验时固定数据、初始化、学习率策略和训练预算。

PyTorch pattern

```
sgd = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
adamw = torch.optim.AdamW(
    model.parameters(), lr=3e-4, weight_decay=1e-2
)
```

易错点

概念易错：把 Adam 的 weight_decay 等同于所有版本/实现中的 L2 正则；同一模型切换优化器却沿用原学习率；忽略 optimizer state 导致恢复训练不一致。

学习率与调度器（learning rate / scheduler）

是什么：学习率控制每次参数更新幅度；调度器按 epoch、step 或验证指标改变学习率。常见有 StepLR、Cosine、OneCycleLR、ReduceLROnPlateau 和 warmup。

为什么需要：学习率通常比模型层数更直接地影响训练是否稳定、收敛速度和最终效果。warmup 可缓解训练初期大更新，退火帮助后期细化。

作用与判断方式：明确 scheduler.step() 应该按 epoch 还是按 optimizer step 调用；打印当前学习率并记录在实验日志中。

PyTorch pattern

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=num_epochs
)
for epoch in range(num_epochs):
    train_one_epoch(...)
    scheduler.step()
```

易错点

代码易错：ReduceLROnPlateau 需要传验证指标；OneCycleLR 常按 batch 调度却误放在 epoch 末；恢复 checkpoint 忘记 scheduler state；学习率变化但日志没记录。

梯度裁剪（gradient clipping）

是什么：梯度裁剪限制梯度范数或单元素幅度，常见是 clip_grad_norm_。它不改变前向 loss，只限制一次更新的最大方向/大小。

为什么需要：RNN、Transformer 或异常 batch 可能出现梯度爆炸；裁剪可以让训练从异常梯度中恢复，但不能替代修复数值或数据问题。

作用与判断方式：混合精度训练时先 unscale 再裁剪；记录裁剪前后的梯度范数，确认阈值不是长期过小。

PyTorch pattern

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

易错点

训练易错：在 GradScaler 尚未 unscale 时直接裁剪；把裁剪当成解决 NaN 的万能药；阈值太小导致模型几乎不学习。

过拟合、欠拟合与泛化（overfitting / underfitting / generalization）

是什么：过拟合是训练集表现好但未见数据表现差；欠拟合是训练集本身也学不好；泛化是模型在新数据上的表现。

为什么需要：模型容量、数据量、噪声、正则化和训练预算共同决定偏差与方差。训练 loss 下降并不保证验证 loss 下降。

作用与判断方式：画 train/valid loss 与核心指标曲线；用小数据过拟合检验实现，用真实验证曲线选择模型和训练时长。

易错点

概念易错：看到训练准确率高就认为模型好；盲目加深模型；只用 dropout 解决数据分布变化；早停依据训练 loss 而不是验证指标。

权重衰减、早停与 checkpoint（weight decay / early stopping / checkpoint）

是什么：weight decay 惩罚大权重；早停在验证指标不再改善时停止；checkpoint 保存可恢复训练或部署的状态。

为什么需要：这些机制控制模型复杂度和训练时间，并防止最后一轮权重不是最佳权重。

作用与判断方式：区分“最佳模型权重”和“最近训练状态”；续训应保存 epoch、model、optimizer、scheduler、scaler、随机状态和配置。

PyTorch pattern

```
checkpoint = {
    "epoch": epoch,
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "scheduler": scheduler.state_dict(),
    "best_metric": best_metric,
}
torch.save(checkpoint, "checkpoint.pt")
```

易错点

代码易错：只保存最后一轮；保存整个 model 对代码路径和类定义强绑定；保存的 best_metric 与 actual best weights 不同步；恢复时忘记 map_location。

随机种子与可复现性（seed / determinism）

是什么：随机种子控制伪随机数生成器的起点；可复现性还涉及 DataLoader worker、CUDA 算子、硬件、软件版本和非确定性实现。

为什么需要：没有可复现性，调参和排错难以判断变化来自代码、数据、随机性还是环境。

作用与判断方式：记录版本、配置、数据版本和 seed；需要严格复现时启用 deterministic 设置，同时接受可能的性能代价。

PyTorch pattern

```
import random, numpy as np, torch
seed = 42
random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

易错点

概念易错：设一个 seed 就保证所有结果位级一致；不同 GPU、驱动或 cuDNN 版本可能仍有差异；为了复现关闭所有随机增强却改变了实验问题。

自动混合精度与 GradScaler（AMP / autocast / GradScaler）

是什么：自动混合精度让适合的算子使用 float16/bfloat16，同时保留部分计算为 float32；GradScaler 通过放大 loss 防止小梯度下溢，并在 step 前处理 inf/NaN。

为什么需要：低精度能减少显存、提高吞吐，尤其适合支持 Tensor Core 的 GPU；但数值敏感算子仍需要更高精度。

作用与判断方式：把 autocast 包住前向和 loss，通常在上下文外 backward；需要查看/裁剪梯度时先 scaler.unscale_(optimizer)。

PyTorch pattern

```
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
for x, y in loader:
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", enabled=use_amp):
        logits = model(x)
        loss = loss_fn(logits, y)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

易错点

AMP 易错：把 backward 放进 autocast；裁剪 scaled gradients；验证时错误地使用 GradScaler；遇到 NaN 只调 scaler 却不检查数据、标签和损失实现。

分布式训练的基本概念（DDP / data parallelism）

是什么：数据并行把不同 batch 分给多个设备，各设备计算局部梯度，再同步梯度并更新相同参数。DDP 通常比单进程多卡 DataParallel 更高效、可扩展。

为什么需要：当单卡显存或吞吐不足时，多卡可以扩大有效 batch 或缩短训练时间。

作用与判断方式：要关注每个进程的 sampler、全局 batch size、学习率缩放、日志只记录一次以及 checkpoint 只由主进程写入。

> **易错点**
> 工程易错：每个进程都 shuffle 同一数据而没有 DistributedSampler；忘记在 epoch 设置 sampler seed；全局 batch 改变却不重新考虑学习率和归一化统计。

7 评估、推理与模型保存

训练阶段追求更新参数，评估阶段追求可信地测量泛化，推理阶段追求稳定、快速、可部署地输出结果。三者的模式和目标不同。

资料提示

推理模式、保存加载和 AMP 推理参考 [P21]-[P24]。

训练模式与推理模式：两个开关分别解决两件事



易错：eval() 不会关闭梯度；no_grad()/inference_mode() 也不会自动切换 Dropout 或 BatchNorm。

model.train() 与 model.eval()（training/evaluation mode）

是什么：model.train() 和 model.eval() 设置模块的 training 标志，影响 Dropout、BatchNorm 等层的行为；它们不直接打开或关闭 autograd。

为什么需要：训练和评估需要不同的随机性和统计量，否则验证指标会抖动或偏离真实部署行为。

作用与判断方式：训练函数开头 model.train()，验证/推理函数开头 model.eval()；如果冻结 BatchNorm 但仍训练其它层，要明确写出策略。

PyTorch pattern

```
model.train()
for x, y in train_loader:
    ...

model.eval()
with torch.inference_mode():
    for x, y in valid_loader:
        logits = model(x)
```

易错点

高频错误：只写 torch.no_grad() 不写 model.eval()；验证后忘记切回 train；把 eval() 当成冻结所有参数；BatchNorm 统计量被验证数据更新。

no_grad 与 inference_mode 的推理开销（evaluation context）

是什么：在不需要梯度的评估/推理中，用 torch.no_grad() 或 torch.inference_mode() 关闭 autograd 记录。后者限制更多，通常更适合纯推理。

为什么需要：减少显存占用和 autograd 开销，并避免把验证结果挂在计算图上。

作用与判断方式：推理代码通常写成 model.eval() 外加 with torch.inference_mode()；如果需要对输出继续参与梯度计算，则不能使用。

易错点

代码易错：把推理输出直接拿去做需要梯度的二次优化；在上下文外保存未 detach 的 CUDA tensor；以为 no_grad 会自动把 tensor 转 CPU。

分类指标：accuracy、precision、recall、F1（classification metrics）

是什么：accuracy 是总体正确率；precision 关注预测为正的样本有多少真正为正；recall 关注真实正类被找回多少；F1 是 precision 和 recall 的调和平均。

为什么需要：类别不平衡时 accuracy 可能掩盖少数类失败；precision/recall 的取舍与业务代价有关。

作用与判断方式：报告 macro、weighted 或 per-class 指标，并说明阈值、平均方式和标签映射。

公式 / 直觉

Precision = TP/(TP+FP)，Recall = TP/(TP+FN)，F1 = 2PR/(P+R)。

易错点

概念易错：把 precision 叫准确率；不说明 positive 类别；F1 高不代表概率校准好；多分类 macro 与 micro 结果不能混为一谈。

混淆矩阵、ROC-AUC 与 PR-AUC（confusion matrix / AUC）

是什么：混淆矩阵按真实类与预测类统计 TP、FP、TN、FN；ROC-AUC 衡量不同阈值下排序能力；PR-AUC 在正类稀少时更有解释力。

为什么需要：指标可从不同角度揭示模型错误，而单个 accuracy 无法说明错在哪里。

作用与判断方式：先保存 logits/probability，再统一选择阈值和计算指标；AUC 不等于在某一个阈值下的准确率。

易错点

评估易错：用离散 argmax 去算 AUC；在验证集选阈值后直接报告测试集 AUC；极度不平衡数据只看 ROC-AUC。

阈值、Top-k 与校准（threshold / top-k / calibration）

是什么：阈值把概率转成决策；Top-k 判断正确类别是否在模型最高的 k 个候选中；校准衡量预测概率与真实发生频率是否一致。

为什么需要：模型排序能力、决策阈值和概率可信度是三个不同问题。医疗、风控和推荐常常需要可解释的置信度而不只是 argmax。

作用与判断方式：阈值应在验证集上选择并固定；线上监控输入分布、置信度和错误率，而不是只看平均 accuracy。

PyTorch pattern

```
prob = torch.softmax(logits, dim=1)
top_prob, top_idx = prob.topk(k=5, dim=1)
binary_pred = (torch.sigmoid(logit) >= threshold)
```

易错点

概念易错：默认 0.5 对所有任务都合理；top-k accuracy 与 top-1 accuracy 混用；把高置信度当成高正确率。

批量推理与后处理（batched inference）

是什么：批量推理把多个输入一起送入模型，提高设备利用率；后处理把 logits 转为概率、类别、框、token 或业务字段。

为什么需要：batch size、CPU 数据准备、GPU 计算和后处理速度共同决定端到端延迟。

作用与判断方式：保证训练与推理预处理一致；按需求平衡吞吐和单请求延迟；大模型推理要注意显存峰值和输出复制。

PyTorch pattern

```
model.eval()
predictions = []
with torch.inference_mode():
    for x in infer_loader:
        logits = model(x.to(device, non_blocking=True))
        predictions.append(logits.cpu())
predictions = torch.cat(predictions)
```

易错点

性能易错：每个样本单独调用模型；循环中频繁 .item() 导致 GPU 同步；保存 GPU tensor 造成显存占用；后处理标签映射与训练时不一致。

state_dict、checkpoint 与 map_location（save / load）

是什么：state_dict() 是参数和持久状态的字典；推理保存通常保存模型 state_dict；续训 checkpoint 还应保存优化器、调度器、epoch、配置和必要的 scaler。

为什么需要：保存状态而不是整个 Python 对象，通常更灵活、依赖更少；续训需要优化器状态中的动量/二阶矩等信息。

作用与判断方式：加载前实例化同样结构的模型，再 load_state_dict()；跨设备加载用 map_location，部署时显式 eval()。

PyTorch pattern

```
torch.save(model.state_dict(), "model_weights.pt")
model.load_state_dict(torch.load("model_weights.pt", map_location=device))
model.eval()

checkpoint = torch.load("checkpoint.pt", map_location=device)
model.load_state_dict(checkpoint["model"])
optimizer.load_state_dict(checkpoint["optimizer"])
```

易错点

代码易错：保存了模型但没有结构定义；加载 state_dict 的 key 不匹配；torch.load 的设备映射不当；部署时加载了 checkpoint 却误把整个字典当 logits。

CPU、GPU 与迁移边界（device placement）

是什么：模型、输入、标签、损失中参与计算的张量必须位于兼容设备；.to(device) 通常返回新对象或新张量。

为什么需要：跨设备拷贝有成本，频繁搬运会拖慢训练并造成 device mismatch。

作用与判断方式：在 batch 进入模型前统一移动；输出需要 numpy 时先 detach().cpu()；不要在循环中反复创建 device。

易错点

代码易错：label 留在 CPU；numpy 与 CUDA tensor 直接转换；推理循环中频繁 CPU/GPU 往返；把 pinned memory 当成自动完成设备迁移。

部署导出与版本契约（export / serving contract）

是什么：部署不仅是导出权重，还包括模型结构、预处理、后处理、标签映射、dtype、设备和版本契约。可按场景考虑 TorchScript、ONNX、ExecuTorch 或直接 Python 服务。

为什么需要：线上输入输出必须与训练时一致，速度、内存和兼容性有时比离线精度更重要。

作用与判断方式：先用同一批固定样例比较训练框架和导出后输出，再做性能压测和异常输入测试。

易错点

部署易错：只导出模型不保存 normalize 参数；导出时训练模式未切换；动态 shape、算子不支持、量化误差和版本差异没有测试。

8 主流网络结构一览

这一章不追求推导每一层，而是建立“什么结构适合什么依赖”的选型地图。真正选型还要结合数据规模、延迟、显存、预训练权重和部署环境。

资料提示

结构来源包括卷积网络、残差网络、序列模型和 Transformer 经典论文 [R1]-[R7]。

结构	核心思想	适合场景	常见易错点
MLP	Linear + 非线性，直接混合特征	表格、低维向量、分类/回归头	输入特征尺度不一致；参数量随维度快速增长
CNN	局部感受野 + 权重共享	图像、局部时空信号	NCHW/NHWC；padding/stride 改变空间尺寸
RNN	按时间步递归更新隐藏状态	短序列、流式场景	长依赖梯度消失；隐藏状态跨 batch 泄漏
LSTM/GRU	门控控制状态读写	需要序列记忆且算力有限	batch_first、pack/pad、hidden shape 混用
Transformer	注意力建立全局依赖	文本、视觉、多模态、长上下文	mask、位置编码、O(T^2) 显存
ViT	把图像切成 patch 当 token	视觉预训练和大规模图像	patch 数、位置编码、输入分辨率变化
预训练 + 微调	复用大数据学到的表示	数据有限、目标任务与预训练相近	冻结/解冻策略；预处理和标签头不匹配

迁移学习、冻结与微调（transfer learning / fine-tuning）

是什么：迁移学习复用源任务学到的参数；冻结表示不更新某些参数；微调是在目标任务数据上继续训练部分或全部网络。

为什么需要：目标数据通常比预训练数据少，复用表示能提高样本效率并减少训练时间。

作用与判断方式：先替换任务头，再决定冻结层和学习率；常给新头较大学习率、预训练层较小学习率。

PyTorch pattern

```
for p in backbone.parameters():
    p.requires_grad = False
for p in model.classifier.parameters():
    p.requires_grad = True

optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3
)
```

易错点

代码易错：冻结后仍把所有参数传给优化器；训练阶段 BatchNorm 仍更新统计量；目标数据预处理与预训练权重要求不一致；解冻后沿用过大学习率破坏预训练表示。

生成式模型、语言模型与自回归（generative / language model / autoregressive）

是什么：生成模型学习数据分布并生成新样本；语言模型预测 token 序列中的下一个 token；自回归生成按序逐步产生输出。

为什么需要：与判别式分类不同，生成任务的目标通常是最大化正确 token 的条件概率，训练可使用 teacher forcing。

作用与判断方式：理解 tokenization、embedding、causal mask、temperature、top-k/top-p 等术语，有助于读懂现代 LLM 训练/推理代码。

易错点

概念易错：训练时使用真实历史 token，推理时却忘记模型要使用自己生成的 token；把 temperature 当成训练学习率；因果 mask 方向写反导致看到未来信息。

量化、蒸馏与参数高效微调（quantization / distillation / PEFT）

是什么：量化用低位宽表示权重或激活；知识蒸馏让小模型学习大模型的软输出；参数高效微调只更新少量附加参数，如 adapter、LoRA。

为什么需要：这些方法降低显存、延迟或训练成本，适合部署与资源受限环境。

作用与判断方式：需要同时验证精度、速度和内存；低位宽误差会放大在敏感层，微调目标要与部署格式一致。

易错点

工程易错：只看参数量不测实际延迟；量化校准数据分布不匹配；LoRA/adapter 保存和加载路径混乱；把蒸馏温度与采样 temperature 混淆。

9 完整 PyTorch 分类示例

下面是一份小而完整的监督分类模板，故意把最容易遗漏的训练、验证、保存和推理边界写全。它可以替换 Dataset、网络和损失函数，作为项目起点。

资料提示

代码模式综合使用 [P5]、[P17]、[P21]、[P23]。

把完整示例当成一条可替换的骨架：数据输入、模型前向、损失计算、参数更新、验证评估、最佳权重保存和新样本推理。先让这条骨架在一个小数据子集上稳定运行，再逐步加入混合精度、调度器和分布式训练。

完整示例的训练闭环



训练时反复执行这条闭环；推理时通常只执行输入 -> 前向传播 -> 输出。

代码阅读顺序

先看模型输出和损失是否匹配，再看训练循环四步，接着看验证模式与指标统计，最后看 checkpoint 和推理契约。这样能把 API 细节放回完整流程中理解。

9.1 训练与验证模板

完整训练/验证模板（1/2）

```
import torch
from torch import nn
from torch.utils.data import DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class Classifier(nn.Module):
    def __init__(self, in_dim=128, n_classes=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, n_classes),
        )

    def forward(self, x):
        return self.net(x)  # logits, shape [B, n_classes]

model = Classifier().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)

def train_one_epoch(model, loader, loss_fn, optimizer, device):
    model.train()
    total_loss, total_correct, total_n = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device).long()
        optimizer.zero_grad(set_to_none=True)
        logits = model(x)
        loss = loss_fn(logits, y)
        loss.backward()
        optimizer.step()

        batch_n = y.size(0)
        total_loss += loss.detach().item() * batch_n
        total_correct += (logits.argmax(dim=1) == y).sum().item()
        total_n += batch_n
    return total_loss / total_n, total_correct / total_n

@torch.inference_mode()
def evaluate(model, loader, loss_fn, device):
    model.eval()
    total_loss, total_correct, total_n = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device).long()
        logits = model(x)
        loss = loss_fn(logits, y)
        batch_n = y.size(0)
        total_loss += loss.item() * batch_n
        total_correct += (logits.argmax(dim=1) == y).sum().item()
        total_n += batch_n
    return total_loss / total_n, total_correct / total_n
```

完整训练/验证模板（2/2）

```
best_valid_acc = -float("inf")
for epoch in range(20):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, loss_fn, optimizer, device
    )
    valid_loss, valid_acc = evaluate(
        model, valid_loader, loss_fn, device
    )
    print(f"{epoch=}: {train_loss=:.4f}, {train_acc=:.3f}, "
          f"{valid_loss=:.4f}, {valid_acc=:.3f}")

    if valid_acc > best_valid_acc:
        best_valid_acc = valid_acc
        torch.save({
            "model": model.state_dict(),
            "optimizer": optimizer.state_dict(),
            "epoch": epoch,
            "valid_acc": valid_acc,
        }, "best.pt")
```

逐段理解这份代码

(1) 模型只输出 logits，不手动 softmax；(2) target 转为 long，符合 CrossEntropyLoss；(3) 每个 batch 先清梯度再 backward/step；(4) 统计 loss 时乘回 batch_n，最后按样本数求平均；(5) evaluate 同时使用 eval 和 inference_mode；(6) 保存验证集最好的权重，而不是盲目保存最后一轮。

9.2 推理代码

批量推理与后处理

```
checkpoint = torch.load("best.pt", map_location=device)
model = Classifier(in_dim=128, n_classes=4).to(device)
model.load_state_dict(checkpoint["model"])
model.eval()

with torch.inference_mode():
    x = x.to(device)
    logits = model(x)
    prob = logits.softmax(dim=-1)
    confidence, pred = prob.max(dim=-1)

pred = pred.cpu().tolist()
confidence = confidence.cpu().tolist()
```

推理前的四项契约

预处理的数值范围和维度顺序必须与训练一致；模型结构和类别数必须一致；类别索引到真实标签的映射必须保存；概率阈值或 top-k 策略必须和业务需求一致。

9.3 如果训练不收敛，按这个顺序排查

顺序	检查	为什么先查它
1	单个 batch 的 x/y shape、dtype、范围、标签分布	输入和标签错了，后面所有调参都没有意义
2	模型输出 shape、是否含 NaN/Inf、logits 语义	损失函数匹配错误最常见且易被忽略
3	loss.backward 后参数 grad 是否为 None/0/NaN	确认计算图、requires_grad 和参数注册正常
4	optimizer 是否包含目标参数，step 是否真的执行	没有更新就不可能学习
5	学习率、归一化、初始化、梯度范数	确认优化过程稳定而不是数值爆炸/停滞
6	训练/验证模式、数据泄漏、评估代码	避免指标虚高或训练/推理行为不一致

10 高频易错点总清单

把容易错误的点集中放在最后，适合作为提交实验、上线推理或代码 review 前的检查表。

10.1 概念层面的错误

误区	正确理解	快速检查
反向传播会更新参数	backward 只计算并累积梯度，step 才更新参数	打印 step 前后某个参数是否改变
loss 越低准确率一定越高	loss 与业务指标相关但不等价	同时看验证 loss、混淆矩阵和目标指标
logits 就是概率	logits 是未归一化分数	检查是否需要 softmax/sigmoid，以及损失是否已内部完成
eval() 就是 no_grad()	eval 改变层行为，no_grad 关闭 autograd	两者通常在验证/推理同时使用
batch size 越大越好	它影响显存、梯度噪声、吞吐和泛化	固定有效 batch 后比较学习率与验证曲线
模型越深越强	容量、数据、优化和正则化需匹配	先用小模型和小数据验证流程
高置信度代表正确	概率需要校准，模型可自信地犯错	检查 reliability diagram / ECE 或错误样例

10.2 训练过程的错误

症状	优先怀疑	修复方向
loss 不下降	标签/损失不匹配、学习率、参数未更新	单 batch 过拟合；检查 logits、grad、optimizer 参数组
loss 变成 NaN/Inf	输入异常、学习率过大、混合精度溢出	检查数据范围、降低 LR、关闭 AMP 定位敏感算子
训练高、验证低	过拟合、泄漏切分、train/eval 模式不一致	验证集独立；加正则/增强；确认 eval
验证指标每次不同	忘记 eval、验证随机增强、未关闭 dropout	固定预处理，eval + inference_mode
GPU 利用率低	数据加载慢、batch 太小、频繁同步	pin_memory、num_workers、增大 batch、少用 item/print
续训后曲线突变	没加载 optimizer/scheduler/scaler 状态	保存并恢复完整 checkpoint
类别少数类很差	只看 accuracy、损失未加权、阈值默认	看 per-class 指标，权重/采样/阈值在验证集调整

10.3 代码层面的错误

代码点	常见错误	更稳妥写法
loss.item()	把它保存后再 backward；或每步 item 导致 GPU 同步	backward 用 loss；日志用 loss.detach().item()，按需降低频率
detach()	以为它复制了数据，或 detach 后继续期待梯度	明确 detach 只切图；需要副本时 clone().detach()
squeeze()	batch=1 时把 batch 维删掉	指定维度，如 squeeze(-1)，并断言 shape
view()	permute 后直接 view，或展平 batch	确认 contiguous，使用 flatten(1)/reshape
state_dict()	只保存模型却无法复现实验	保存配置、类别映射、预处理与训练状态
DataLoader	train/valid 共享随机 transform；worker 导致复现差	拆分 transform，配置 worker seed 与 sampler
混合精度	scaled gradient 直接裁剪	先 scaler.unscale_(optimizer)，再 clip
设备迁移	模型在 GPU，标签/输入在 CPU	统一在 batch 边界 .to(device) 并检查 .device

最终提交前的 12 项检查

输入 shape/dtype/range；标签编码；模型输出语义；损失匹配；train/eval；zero_grad；grad 是否有效；optimizer 参数组；学习率日志；验证无泄漏；best checkpoint；推理预处理和标签映射。

附录 PyTorch 高频 API 速查与参考资料

把术语和代码入口收在一起，适合在项目中快速查阅。

A. 高频 API 速查

目的	常用 API	注意
定义模型	nn.Module, nn.Linear, nn.Conv2d, nn.Sequential	层要注册在 Module 中
加载数据	Dataset, DataLoader, transforms	训练/验证 transform 分开
训练	model.train(), loss.backward(), optimizer.step()	先 zero_grad，后 step
验证	model.eval(), torch.inference_mode()	eval 与 no_grad 是两个概念
移动设备	tensor.to(device), model.to(device)	参与同一运算的 tensor 设备要一致
保存	model.state_dict(), optimizer.state_dict(), torch.save	续训保存完整状态
调度	torch.optim.lr_scheduler	确认按 epoch 还是按 step 调用
混合精度	torch.autocast, torch.amp.GradScaler	裁剪前 unscale
排错	assert, named_parameters, grad.norm, anomaly detection	先缩小到单 batch/小模型

B. 参考资料与检索来源

[P1] PyTorch Tutorials - Automatic Differentiation with torch.autograd
https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

[P2] PyTorch Tutorials - Optimization
https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[P3] PyTorch Tutorials - Tensors
https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

[P4] PyTorch Docs - inference_mode
https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad_mode.inference_mode.html

[P5] PyTorch Tutorials - Datasets & DataLoaders
https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html

[P6] PyTorch Tutorials - Writing Custom Datasets, DataLoaders and Transforms
https://docs.pytorch.org/tutorials/beginner/data_loading_tutorial.html

[P7] PyTorch Docs - torch.nn.Module
https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

[P8] PyTorch Docs - Dropout
https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html

[P9] PyTorch Docs - BatchNorm2d
https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html

[P10] PyTorch Docs - MultiheadAttention
https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html

[P11] PyTorch Docs - Embedding
https://docs.pytorch.org/docs/stable/generated/torch.nn.Embedding.html

[P12] PyTorch Docs - Conv2d
https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

[P13] PyTorch Docs - Linear
https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html

[P14] PyTorch Docs - CrossEntropyLoss
https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[P15] PyTorch Docs - BCEWithLogitsLoss
https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html

[P16] PyTorch Docs - MSELoss
https://docs.pytorch.org/docs/stable/generated/torch.nn.MSELoss.html

[P17] PyTorch Tutorials - Quickstart
https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

[P18] PyTorch Docs - torch.optim
https://docs.pytorch.org/docs/stable/optim.html

[P19] PyTorch Docs - Reproducibility
https://docs.pytorch.org/docs/stable/notes/randomness.html

[P20] PyTorch Tutorials - Saving and Loading Models
https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html

[P21] PyTorch Tutorials - Automatic Mixed Precision Recipe
https://docs.pytorch.org/tutorials/recipes/recipes/amp_recipe.html

[P22] PyTorch Docs - Automatic Mixed Precision
https://docs.pytorch.org/docs/stable/amp.html

[P23] PyTorch Tutorials - Save & Load Model
https://docs.pytorch.org/tutorials/beginner/basics/saveloadrun_tutorial.html

[P24] PyTorch Tutorials - Model Evaluation and Inference
https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[R1] LeCun, Bengio, Hinton. Deep learning. Nature, 2015.
https://doi.org/10.1038/nature14539

[R2] He et al. Deep Residual Learning for Image Recognition, CVPR 2016.
https://doi.org/10.1109/CVPR.2016.90

[R3] Vaswani et al. Attention Is All You Need, NeurIPS 2017.
https://arxiv.org/abs/1706.03762

[R4] Kingma and Ba. Adam: A Method for Stochastic Optimization.
https://arxiv.org/abs/1412.6980

[R5] Ioffe and Szegedy. Batch Normalization.
https://arxiv.org/abs/1502.03167

[R6] Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting.
https://jmlr.org/papers/v15/srivastava14a.html

[R7] Goodfellow, Bengio, Courville. Deep Learning.
https://www.deeplearningbook.org/

版本说明

本文档内容按 2026-09-21 的检索结果整理，优先采用 PyTorch 官方教程/文档和经典论文。框架 API、默认参数与硬件支持会随版本变化；使用前请对照当前环境中的官方文档和实际测试结果。